

CS 6160 Cryptology: Block Ciphers

Shibam Ghosh

September 3, 2026

Department of CSE, IIT Hyderabad



Discussion on Homework

What is AES-NI?

- ▶ **AES-NI (AES New Instructions)** is a set of hardware instructions designed to accelerate AES operations.
- ▶ Instead of implementing AES transformations using many software instructions, AES-NI performs an entire AES round using a small number of CPU instructions.
- ▶ The instructions are available through C/C++ intrinsics, for example:

```
#include <immintrin.h>
```

- ▶ The most important encryption intrinsics are:

```
    > _mm_aesenc_si128
```

```
    > _mm_aesenclast_si128
```

- ▶ The corresponding decryption intrinsics include:

```
    > _mm_aesdec_si128
```

```
    > _mm_aesdeclast_si128
```

What is AES-NI?

- ▶ **AES-NI (AES New Instructions)** is a set of hardware instructions designed to accelerate AES operations.
- ▶ Instead of implementing AES transformations using many software instructions, AES-NI performs an entire AES round using a small number of CPU instructions.
- ▶ The instructions are available through C/C++ intrinsics, for example:

```
#include <immintrin.h>
```

- ▶ The most important encryption intrinsics are:

- ▷ `_mm_aesenc_si128`

- ▷ `_mm_aesenclast_si128`

- ▶ The corresponding decryption intrinsics include:

- ▷ `_mm_aesdec_si128`

- ▷ `_mm_aesdeclast_si128`

What is AES-NI?

- ▶ **AES-NI (AES New Instructions)** is a set of hardware instructions designed to accelerate AES operations.
- ▶ Instead of implementing AES transformations using many software instructions, AES-NI performs an entire AES round using a small number of CPU instructions.
- ▶ The instructions are available through C/C++ intrinsics, for example:

```
#include <immintrin.h>
```

- ▶ The most important encryption intrinsics are:

```
▷ _mm_aesenc_si128
```

```
▷ _mm_aesenclast_si128
```

- ▶ The corresponding decryption intrinsics include:

```
▷ _mm_aesdec_si128
```

```
▷ _mm_aesdeclast_si128
```

What is AES-NI?

- ▶ **AES-NI (AES New Instructions)** is a set of hardware instructions designed to accelerate AES operations.
- ▶ Instead of implementing AES transformations using many software instructions, AES-NI performs an entire AES round using a small number of CPU instructions.
- ▶ The instructions are available through C/C++ intrinsics, for example:

```
#include <immintrin.h>
```

- ▶ The most important encryption intrinsics are:

- ▷ `_mm_aesenc_si128`
- ▷ `_mm_aesenclast_si128`

- ▶ The corresponding decryption intrinsics include:

- ▷ `_mm_aesdec_si128`
- ▷ `_mm_aesdeclast_si128`

What is AES-NI?

- ▶ **AES-NI (AES New Instructions)** is a set of hardware instructions designed to accelerate AES operations.
- ▶ Instead of implementing AES transformations using many software instructions, AES-NI performs an entire AES round using a small number of CPU instructions.
- ▶ The instructions are available through C/C++ intrinsics, for example:

```
#include <immintrin.h>
```

- ▶ The most important encryption intrinsics are:
 - ▷ `_mm_aesenc_si128`
 - ▷ `_mm_aesenclast_si128`
- ▶ The corresponding decryption intrinsics include:
 - ▷ `_mm_aesdec_si128`
 - ▷ `_mm_aesdeclast_si128`

AES-NI: What Does Each Instruction Do?

- ▶ `_mm_aesenc_si128(state, roundkey)` Performs one round AES encryption round:

SubBytes → ShiftRows → MixColumns → AddRoundKey.

- ▶ `_mm_aesenclast_si128(state, roundkey)` Performs the final AES encryption round:

SubBytes → ShiftRows → AddRoundKey.

- ▶ `_mm_aesdec_si128(state, roundkey)` Performs one AES decryption round:

InvShiftRows → InvSubBytes → InvMixColumns → AddRoundKey.

- ▶ `_mm_aesdeclast_si128(state, roundkey)` Performs the final AES decryption round:

InvShiftRows → InvSubBytes → AddRoundKey.

AES-NI: What Does Each Instruction Do?

- ▶ `_mm_aesenc_si128(state, roundkey)` Performs one round AES encryption round:
SubBytes → ShiftRows → MixColumns → AddRoundKey.
- ▶ `_mm_aesenclast_si128(state, roundkey)` Performs the final AES encryption round:
SubBytes → ShiftRows → AddRoundKey.
- ▶ `_mm_aesdec_si128(state, roundkey)` Performs one AES decryption round:
InvShiftRows → InvSubBytes → InvMixColumns → AddRoundKey.
- ▶ `_mm_aesdeclast_si128(state, roundkey)` Performs the final AES decryption round:
InvShiftRows → InvSubBytes → AddRoundKey.

AES-NI: What Does Each Instruction Do?

- ▶ `_mm_aesenc_si128(state, roundkey)` Performs one round AES encryption round:
SubBytes $\rightarrow$ ShiftRows $\rightarrow$ MixColumns $\rightarrow$ AddRoundKey.
- ▶ `_mm_aesenclast_si128(state, roundkey)` Performs the final AES encryption round:
SubBytes $\rightarrow$ ShiftRows $\rightarrow$ AddRoundKey.
- ▶ `_mm_aesdec_si128(state, roundkey)` Performs one AES decryption round:
InvShiftRows $\rightarrow$ InvSubBytes $\rightarrow$ InvMixColumns $\rightarrow$ AddRoundKey.
- ▶ `_mm_aesdeclast_si128(state, roundkey)` Performs the final AES decryption round:
InvShiftRows $\rightarrow$ InvSubBytes $\rightarrow$ AddRoundKey.

Homework: AES Software vs. AES-NI

Programming Task

Implement the specified AES-like cipher in **C** or **C++** in two different ways:

1. Conventional software implementation

- ▷ Explicitly implement the AES transformations.
- ▷ Implement both encryption and decryption.

2. AES-NI implementation

- ▷ Use AES-NI intrinsics.
- ▷ Implement both encryption and decryption.

Both implementations must produce exactly the same ciphertext for the same plaintext and key.

Homework: AES Software vs. AES-NI

Programming Task

Implement the specified AES-like cipher in **C or C++** in two different ways:

1. Conventional software implementation

- ▷ Explicitly implement the AES transformations.
- ▷ Implement both encryption and decryption.

2. AES-NI implementation

- ▷ Use AES-NI intrinsics.
- ▷ Implement both encryption and decryption.

Both implementations must produce exactly the same ciphertext for the same plaintext and key.

Homework: AES Software vs. AES-NI

Programming Task

Implement the specified AES-like cipher in **C** or **C++** in two different ways:

1. **Conventional software implementation**

- ▷ Explicitly implement the AES transformations.
- ▷ Implement both encryption and decryption.

2. **AES-NI implementation**

- ▷ Use AES-NI intrinsics.
- ▷ Implement both encryption and decryption.

Both implementations must produce exactly the same ciphertext for the same plaintext and key.

Homework: AES Software vs. AES-NI

Programming Task

Implement the specified AES-like cipher in **C** or **C++** in two different ways:

1. **Conventional software implementation**

- ▷ Explicitly implement the AES transformations.
- ▷ Implement both encryption and decryption.

2. **AES-NI implementation**

- ▷ Use AES-NI intrinsics.
- ▷ Implement both encryption and decryption.

Both implementations must produce exactly the same ciphertext for the same plaintext and key.

Homework: Modified Key Schedule

You are **not required to implement the standard AES key schedule.**

Let K be the given 128-bit master key.

Use it directly as the **pre-whitening key**:

$$k_{-1} = K.$$

The remaining round keys are generated recursively:

$$k_i = \text{ShiftRows}(k_{i-1}) \oplus \text{RC}_i, \quad 0 \leq i \leq 9.$$



- ▶ RC_i is the i -th round constant, represented as a 128-bit value.
- ▶ The generated keys are

Homework: Modified Key Schedule

You are **not required to implement the standard AES key schedule.**

Let K be the given 128-bit master key.

Use it directly as the **pre-whitening key**:

$$k_{-1} = K.$$

The remaining round keys are generated recursively:

$$k_i = \text{ShiftRows}(k_{i-1}) \oplus \text{RC}_i, \quad 0 \leq i \leq 9.$$



- ▶ RC_i is the i -th round constant, represented as a 128-bit value.
- ▶ The generated keys are

Homework: Modified Key Schedule

You are **not required to implement the standard AES key schedule.**

Let K be the given 128-bit master key.

Use it directly as the **pre-whitening key**:

$$k_{-1} = K.$$

The remaining round keys are generated recursively:

$$k_i = \text{ShiftRows}(k_{i-1}) \oplus \text{RC}_i, \quad 0 \leq i \leq 9.$$



- ▶ RC_i is the i -th round constant, represented as a 128-bit value.
- ▶ The generated keys are

Homework: Modified Key Schedule

You are **not required to implement the standard AES key schedule.**

Let K be the given 128-bit master key.

Use it directly as the **pre-whitening key**:

$$k_{-1} = K.$$

The remaining round keys are generated recursively:

$$k_i = \text{ShiftRows}(k_{i-1}) \oplus \text{RC}_i, \quad 0 \leq i \leq 9.$$

- ▶ RC_i is the i -th round constant, represented as a 128-bit value.
- ▶ The generated keys are

Generating the Modified Round Keys with AES-NI

Recall that the modified key schedule is

$$k_{-1} = K, \quad k_i = \text{ShiftRows}(k_{i-1}) \oplus RC_i.$$

Implementing ShiftRows with AES-NI

The AES state is stored in a 128-bit register. We can rearrange its 16 bytes using:

```
_mm_shuffle_epi8
```

This instruction uses a **shuffle mask** to specify where each output byte comes from.

Generating the Modified Round Keys with AES-NI

Recall that the modified key schedule is

$$k_{-1} = K, \quad k_i = \text{ShiftRows}(k_{i-1}) \oplus RC_i.$$

Implementing ShiftRows with AES-NI

The AES state is stored in a 128-bit register. We can rearrange its 16 bytes using:

`_mm_shuffle_epi8`

This instruction uses a **shuffle mask** to specify where each output byte comes from.

Generating the Modified Round Keys with AES-NI

Recall that the modified key schedule is

$$k_{-1} = K, \quad k_i = \text{ShiftRows}(k_{i-1}) \oplus RC_i.$$

Implementing ShiftRows with AES-NI

The AES state is stored in a 128-bit register. We can rearrange its 16 bytes using:

`_mm_shuffle_epi8`

This instruction uses a **shuffle mask** to specify where each output byte comes from.

AES-NI Byte Shuffle for ShiftRows

The shuffle mask can be constructed as:

```
const __m128i shiftrows_mask =  
    _mm_setr_epi8(  
        0,  5, 10, 15,  
        4,  9, 14,  3,  
        8, 13,  2,  7,  
        12, 1,  6, 11  
    );
```

Then compute the next round key as:

```
__m128i shifted =  
    _mm_shuffle_epi8(  
        round_key,  
        shiftrows_mask  
    );
```

AES-NI Byte Shuffle for ShiftRows

The shuffle mask can be constructed as:

```
const __m128i shiftrows_mask =  
    _mm_setr_epi8(  
        0,  5, 10, 15,  
        4,  9, 14,  3,  
        8, 13,  2,  7,  
        12, 1,  6, 11  
    );
```

Then compute the next round key as:

```
__m128i shifted =  
    _mm_shuffle_epi8(  
        round_key,  
        shiftrows_mask  
    );
```

AES-NI Byte Shuffle for ShiftRows

The shuffle mask can be constructed as:

```
const __m128i shiftrows_mask =  
    _mm_setr_epi8(  
        0,  5, 10, 15,  
        4,  9, 14,  3,  
        8, 13,  2,  7,  
        12, 1,  6, 11  
    );
```

Then compute the next round key as:

```
__m128i shifted =  
    _mm_shuffle_epi8(  
        round_key,  
        shiftrows_mask  
    );
```

AES-NI Byte Shuffle for ShiftRows

The shuffle mask can be constructed as:

```
const __m128i shiftrows_mask =  
    _mm_setr_epi8(  
        0,  5, 10, 15,  
        4,  9, 14,  3,  
        8, 13,  2,  7,  
        12, 1,  6, 11  
    );
```

Then compute the next round key as:

```
__m128i shifted =  
    _mm_shuffle_epi8(  
        round_key,  
        shiftrows_mask  
    );
```

Performance Measurement: Throughput

Suppose N is total number of bytes processed and T is total execution time in seconds. Then

$$\text{Throughput} = \frac{N}{T}$$

and can be reported in KB/s.

Recommended Procedure:

1. Generate all round keys.
2. Prepare a sufficiently large input.
3. Start the timer.
4. Encrypt or decrypt all blocks.
5. Stop the timer and Compute N/T .

Do **not** include key generation in the main encryption/decryption timing.

Performance Measurement: Throughput

Suppose N is total number of bytes processed and T is total execution time in seconds. Then

$$\text{Throughput} = \frac{N}{T}$$

and can be reported in KB/s.

Recommended Procedure:

1. Generate all round keys.
2. Prepare a sufficiently large input.
3. Start the timer.
4. Encrypt or decrypt all blocks.
5. Stop the timer and Compute N/T .

Do **not** include key generation in the main encryption/decryption timing.

Performance Measurement: Throughput

Suppose N is total number of bytes processed and T is total execution time in seconds. Then

$$\text{Throughput} = \frac{N}{T}$$

and can be reported in KB/s.

Recommended Procedure:

1. Generate all round keys.
2. Prepare a sufficiently large input.
3. Start the timer.
4. Encrypt or decrypt all blocks.
5. Stop the timer and Compute N/T .

Do **not** include key generation in the main encryption/decryption timing.

Performance Measurement: Cycles per Byte

Why Cycles per Byte?

Execution time depends on the processor frequency. Cycles per byte provides a more processor-oriented performance metric.

- ▶ C_{start} = CPU cycle counter before processing;
- ▶ C_{end} = CPU cycle counter after processing;
- ▶ N = total number of bytes processed.

Therefore,

$$\text{CPB} = \frac{C_{\text{end}} - C_{\text{start}}}{N}.$$

For N_b AES blocks, $N = 16N_b$, hence

$$\text{CPB} = \frac{C}{16N_b}.$$

Performance Measurement: Cycles per Byte

Why Cycles per Byte?

Execution time depends on the processor frequency. Cycles per byte provides a more processor-oriented performance metric.

- ▶ C_{start} = CPU cycle counter before processing;
- ▶ C_{end} = CPU cycle counter after processing;
- ▶ N = total number of bytes processed.

Therefore,

$$\text{CPB} = \frac{C_{\text{end}} - C_{\text{start}}}{N}.$$

For N_b AES blocks, $N = 16N_b$, hence

$$\text{CPB} = \frac{C}{16N_b}.$$

Performance Measurement: Cycles per Byte

Why Cycles per Byte?

Execution time depends on the processor frequency. Cycles per byte provides a more processor-oriented performance metric.

- ▶ C_{start} = CPU cycle counter before processing;
- ▶ C_{end} = CPU cycle counter after processing;
- ▶ N = total number of bytes processed.

Therefore,

$$\text{CPB} = \frac{C_{\text{end}} - C_{\text{start}}}{N}.$$

For N_b AES blocks, $N = 16N_b$, hence

$$\text{CPB} = \frac{C}{16N_b}.$$

Performance Measurement: Cycles per Byte

Why Cycles per Byte?

Execution time depends on the processor frequency. Cycles per byte provides a more processor-oriented performance metric.

- ▶ C_{start} = CPU cycle counter before processing;
- ▶ C_{end} = CPU cycle counter after processing;
- ▶ N = total number of bytes processed.

Therefore,

$$\text{CPB} = \frac{C_{\text{end}} - C_{\text{start}}}{N}.$$

For N_b AES blocks, $N = 16N_b$, hence

$$\text{CPB} = \frac{C}{16N_b}.$$

Measuring CPU Cycles

On an x86 processor, you may use the timestamp counter.

```
#include <x86intrin.h>

uint64_t start = __rdtsc();
..... /* Encrypt or decrypt all blocks */
uint64_t end = __rdtsc();

uint64_t cycles = end - start;

double cpb = (double)cycles / total_bytes;
```

- ▶ Process a sufficiently large amount of data.
- ▶ Repeat the experiment several times.
- ▶ Report the average or median result.

Measuring CPU Cycles

On an x86 processor, you may use the timestamp counter.

```
#include <x86intrin.h>
```

```
uint64_t start = __rdtsc();
```

```
..... /* Encrypt or decrypt all blocks */
```

```
uint64_t end = __rdtsc();
```

```
uint64_t cycles = end - start;
```

```
double cpb = (double)cycles / total_bytes;
```

- ▶ Process a sufficiently large amount of data.
- ▶ Repeat the experiment several times.
- ▶ Report the average or median result.

Measuring CPU Cycles

On an x86 processor, you may use the timestamp counter.

```
#include <x86intrin.h>
```

```
uint64_t start = __rdtsc();
```

```
..... /* Encrypt or decrypt all blocks */
```

```
uint64_t end = __rdtsc();
```

```
uint64_t cycles = end - start;
```

```
double cpb = (double)cycles / total_bytes;
```

- ▶ Process a sufficiently large amount of data.
- ▶ Repeat the experiment several times.
- ▶ Report the average or median result.

Homework: What to Submit

1. Complete C/C++ source code

- ▷ Software encryption and decryption.
- ▷ AES-NI encryption and decryption.

2. Compilation and execution instructions

- ▷ Compiler version.
- ▷ Compiler flags.
- ▷ Required architecture options.

3. Short report

- ▷ Machine and CPU specification.
- ▷ Description of the implementation.
- ▷ Correctness verification.
- ▷ Throughput results.
- ▷ Cycles-per-byte results.
- ▷ Brief discussion of the results.

Homework: What to Submit

1. Complete C/C++ source code

- ▷ Software encryption and decryption.
- ▷ AES-NI encryption and decryption.

2. Compilation and execution instructions

- ▷ Compiler version.
- ▷ Compiler flags.
- ▷ Required architecture options.

3. Short report

- ▷ Machine and CPU specification.
- ▷ Description of the implementation.
- ▷ Correctness verification.
- ▷ Throughput results.
- ▷ Cycles-per-byte results.
- ▷ Brief discussion of the results.

Homework: What to Submit

1. Complete C/C++ source code

- ▷ Software encryption and decryption.
- ▷ AES-NI encryption and decryption.

2. Compilation and execution instructions

- ▷ Compiler version.
- ▷ Compiler flags.
- ▷ Required architecture options.

3. Short report

- ▷ Machine and CPU specification.
- ▷ Description of the implementation.
- ▷ Correctness verification.
- ▷ Throughput results.
- ▷ Cycles-per-byte results.
- ▷ Brief discussion of the results.

Homework: What to Submit

Prepare a table similar to the following.

Implementation	1 KB	2 KB	4 KB	32 KB	64 KB
Throughput (MB/s)					
Software Encryption					
AES-NI Encryption					
Software Decryption					
AES-NI Decryption					
Cycles per Byte (CPB)					
Software Encryption					
AES-NI Encryption					
Software Decryption					
AES-NI Decryption					



Thank You for your
attention!

Any questions?